Instructions for Lab Submission

1. **File Naming and Submission:**
   a. Submit your solution as a **ZIP file** named `lab<number>_<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
   b. Ensure that the ZIP file contains:
      i. C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
      ii. No unnecessary files (such as executables, temporary files, or editor configs).
   c. Do not submit **TEXT(.txt)** or **EXE(.exe)** files.
2. **Code Organization:**
   a. Write each task in its own `.cpp` file (as specified above).
   b. Ensure that your code is modular and logically separated into functions when specified.
3. **Code Neatness:**
   a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
   b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
   c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. **Code Compilation and Functionality:**
   a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
   b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. **Additional Instructions:**
   a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
   b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
   c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. **Deadline:**
   a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted
   b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. **Plagiarism Policy:**
   a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

1. Functions

A **function** is a named block of code that performs a specific task. Functions help divide a program into smaller, reusable parts.

For example:

```
int mul(int n, int m) {
    return n * m;
}
```

This function receives an integer `n` and `m`, calculates its product, and returns the result.

A function can be called whenever its task is needed:

```
int result = mul(5, 4);
cout << result;   // 20
```

The main benefits of functions are:

- Code reusability.
- Better organization and readability.
- Easier testing and debugging.
- Reduced repetition.

2. Function Declaration and Definition

A **function declaration** tells the compiler the function's name, return type, and parameters.

```
int mul(int n, int m)
```

A **function definition** contains the actual statements that perform the task.

```
int mul(int n, int m) {
    return n * m;
}
```

A complete example:

```
#include <iostream>
using namespace std;

int mul(int n, int m);

int main() {
    cout << mul(5, 4);
    return 0;
}

int mul(int n, int m) {
    return n * m;
}
```

Output:

```
20
```

A function must be declared before it is called if its definition appears later in the program. Alternatively, its complete definition can appear before `main()`.

3. Parameters and Arguments

Parameters are variables listed in a function's definition. **Arguments** are the actual values passed to the function when it is called.

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(5, 3);
    cout << result;

    return 0;
}
```

In this example:

- `a` and `b` are parameters.
- `5` and `3` are arguments.
- The function returns `8`.

A function can accept zero, one, or multiple parameters, depending on the task it performs.

For example, a function with no parameters:

```
void greet() {
    cout << "Hello!";
}
```

Calling the function:

```
greet();
```

4. Pass-by-Value and Pass-by-Reference

When arguments are passed to functions, the way they are passed determines whether changes made inside the function affect the original variables.

Pass-by-Value

In pass-by-value, a copy of the argument is passed to the function. Changes made to the parameter do not affect the original variable.

```
#include <iostream>
using namespace std;

void change(int n) {
    n = 100;
}

int main() {
    int number = 10;

    change(number);

    cout << number;

    return 0;
}
```

Output:

```
10
```

In this example, `n` is a separate copy of `number`. Changing `n` inside the function does not change `number`.

Pass-by-Reference

In pass-by-reference, the parameter refers to the original variable. Changes made to the parameter directly affect the original variable.

To pass an argument by reference, use the reference operator `&` in the function parameter declaration.

```
#include <iostream>
using namespace std;

void change(int &n) {
    n = 100;
}

int main() {
    int number = 10;

    change(number);

    cout << number;

    return 0;
}
```

Output:

```
100
```

In this example, `n` refers to the original variable `number`. Therefore, changing `n` also changes `number`.

Comparison

Feature	**Pass-by-Value**	**Pass-by-Reference**
Parameter	Receives a copy	Refers to original variable
Changes original variable?	No	Yes
Parameter syntax	`int n`	`int &n`
Typical use	When original data should remain unchanged	When original data needs modification

Important Reminders

- Pass-by-value does not modify the original variable through the parameter.
- Pass-by-reference allows a function to modify the original variable.
- The `&` symbol in a reference parameter declares a reference; it is different from using `&` to obtain an address.
- A reference parameter must be initialized through the argument supplied when the function is called.
- Arrays passed using parameters such as `int arr[]` can have their original elements modified inside the function.

4. Return Types and the `return` Statement

The **return type** specifies the type of value a function gives back.

Common return types include:

- `int`: returns an integer.
- `double`: returns a decimal number.
- `char`: returns a character.
- `bool`: returns `true` or `false`.
- `void`: does not return a value.

Example of a function that returns a value:

```
int multiply(int a, int b) {
    return a * b;
}
```

Example of a `void` function:

```
void display() {
    cout << "Welcome!";
}
```

A returned value can be stored in a variable or used directly:

```
int result = multiply(4, 5);

cout << multiply(3, 6);
```

The `return` statement sends a value back to the calling code and ends the function's execution.

5. Void and Value-Returning Functions

Functions can be classified according to whether they return a value.

Void function: Performs a task without returning a value.

```
void printSum(int a, int b) {
    cout << a + b << endl;
}
```

Call:

```
printSum(4, 6);
```

Output:

```
10
```

Value-returning function: Calculates and returns a result.

```
int getSum(int a, int b) {
    return a + b;
}
```

Call:

```
int sum = getSum(4, 6);
cout << sum;
```

Output:

```
10
```

Use a `void` function when a value does not need to be returned. Use a value-returning function when the result is needed by the calling code.

6. Default Parameters

A **default parameter** is a parameter that has a predefined value. If the caller does not provide an argument for that parameter, the default value is used.

For example:

```
void greet(int times = 1) {
    for (int i = 0; i < times; i++) {
        cout << "Hello!" << endl;
    }
}
```

The function can be called in two ways:

```
greet();      // Prints Hello! once
greet(3);     // Prints Hello! three times
```

Another example:

```
int power(int n, int exponent = 2) {
    int result = 1;

    for (int i = 0; i < exponent; i++) {
        result *= n;
    }

    return result;
}
```

Calls:

```
cout << power(4);      // 16
cout << power(4, 3);   // 64
```

In the first call, the default exponent of `2` is used. In the second call, the supplied exponent of `3` is used.

Important rules:

- Default values are specified in the function declaration or definition, not repeatedly in both.
- Parameters with default values must appear after parameters without default values.
- Arguments are matched to parameters from left to right.

8. Local and Global Variables

A **local variable** is declared inside a function or block and is generally accessible only within that scope.

```
void display() {
    int number = 10;
    cout << number;
}
```

The variable `number` is local to `display()`.

A **global variable** is declared outside all functions and can be accessed by functions within its scope.

```
#include <iostream>
using namespace std;

int number = 10;

void display() {
    cout << number;
}

int main() {
    display();
    return 0;
}
```

Output:

```
10
```

Local variables are useful for temporary calculations, while global variables can be shared between functions when necessary. However, global variables should be used carefully because changes to them can affect multiple parts of a program.

9. Passing Arrays to Functions

Arrays can be passed to functions so that the function can process their elements.

For a one-dimensional array, the function commonly receives the array and its size as separate parameters.

```
void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}
```

Calling the function:

```
int numbers[5] = {2, 4, 6, 8, 10};

displayArray(numbers, 5);
```

Output:

```
2 4 6 8 10
```

A function can also modify the original array elements:

```
void increaseArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] += 1;
    }
}
```

After calling `increaseArray(numbers, 5)`, each element of the original array is increased by one.

When an array is passed to a function in this way, changes to its elements affect the original array.

11. Important Reminders

- A function is a reusable block of code that performs a specific task.
- A function declaration specifies its return type, name, and parameters.
- A function definition contains the statements that perform its task.
- Parameters are variables in a function definition; arguments are values passed during a function call.
- The return type specifies the kind of value a function returns.
- A `void` function does not return a value.
- The `return` statement sends a value back to the caller and ends the function's execution.
- Default parameters provide values when corresponding arguments are omitted.
- Parameters with default values must follow parameters without default values.
- Local variables are limited to their scope, while global variables can be accessed by functions within their scope.
- Arrays can be passed to functions along with their sizes.
- Changes to array elements made inside a function affect the original array when the array is passed in this way.
- Functions can contain loops, conditional statements, and other function calls.

Lab Tasks

Task 1: Square and Cube

Problem Description

A mathematics program needs to calculate the square and cube of a number entered by the user.

Write a program that takes an integer n as input from user and uses two separate functions: `square()` to calculate and return the square of the number, and `cube()` to calculate and return its cube. Display both results in `main()`.

Sample Output

```
Enter a number: 4
Square = 16
Cube = 64
```

Task 2: The Number Table

Problem Description

Write a program that displays the multiplication table of a number from 1 up to that number. For example, if the user enters 5, the program should display the table of 5 from 5×1 to 5×5.

Implement a function `printTable()` that accepts the number as a parameter and displays its multiplication table. The function must have a default parameter value of 10.

In `main()`, take an integer as input. If the entered number is greater than 10, call `printTable()` without passing an argument so that the default value is used. Otherwise, pass the entered number to the function.

Sample Output 1

```
Enter a number: 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
```

Sample Output 2

```
Enter a number: 15
10 x 1 = 10
10 x 2 = 20
...
10 x 10 = 100
```

Task 3: Electricity Bill Calculation Using Functions

Problem Description

A colony has `n` houses, and each house must pay its electricity bill based on the units consumed.

Write a C++ program that takes the number of houses `n` as input and uses the following functions:

- `inputUnits()`: Takes electricity units for all houses and stores them in an array.
- `calculateBills()`: Calculates each house's bill using the rates below, applies the discount, and stores the final amounts in a separate array.
- `displayBills()`: Displays the final electricity bill for each house.

Electricity Rates:

Units Consumed	Rate per Unit
First 100 units	Rs. 10
Next 100 units (101–200)	Rs. 17.5
Above 200 units	Rs. 26.5

Apply a **7.5% discount** if the units consumed are even; otherwise, apply a **2.5% discount**. Rates must be applied progressively according to the unit ranges.

Use the following function signatures:

```
void inputUnits(int units[], int n);

void calculateBills(int units[], double bills[], int n);

void displayBills(double bills[], int n);
```

All input, calculations, and output must be performed through their respective functions. The `main()` function should only take `n` as input, declare the arrays, and call the functions.

Sample Output

```
Enter number of houses: 3
Enter units for House 1: 80
Enter units for House 2: 150
Enter units for House 3: 250

Electricity Bills:
House 1: Rs. 740
House 2: Rs. 1828.13
House 3: Rs. 2836.88
```

Task 4: Digit Analysis Using Reference Parameters

Problem Description

Write a function `analyzeDigits()` that receives a positive integer `n`, an integer variable `count`, an integer variable `sum`, and a double variable `average`.

The function must use a loop to count the digits of `n`, calculate their sum, and determine their average **without using the modulus operator (%)**.

The parameters `count`, `sum`, and `average` must be passed by reference so that the function can update their values, while `n` must be passed by value.

The function must have the following signature:

```
void analyzeDigits(int n, int &count, int &sum, double &average);
```

In `main()`, take a positive integer as input, call the function, and display the number of digits, sum of digits, and average of digits.

Restriction: The modulus operator (`%`) is strictly prohibited throughout the program.

Sample Output

```
Enter a positive integer: 5832
Number of digits = 4
Sum of digits = 18
Average of digits = 4.5
```

Task 5: Complete the Insertion Sort and Display Functions

Problem Description

A program sorts an array of integers using the Insertion Sort algorithm. The function `insertionSort()` receives an integer array, its size, a flag indicating whether the array should be sorted in ascending or descending order, and a starting position that has a default value of 1.

Additionally, the program uses a separate function `displayArray()` that receives an integer array and its size to display all the elements of the array.

Complete the following code by filling in the missing parameter data types and names, completing the optional parameter, and writing the missing statements in both functions.

The program must satisfy the following requirements:

- Implement the Insertion Sort algorithm to arrange the array elements in the specified order.
- If the ascending flag is `true`, sort the array in ascending order. Otherwise, sort it in descending order.
- Complete the `displayArray()` function to print all array elements separated by spaces.
- Use the `displayArray()` function in `main()` to display the array before and after sorting.

```
#include <iostream>
using namespace std;

// declare display Array function here

void insertionSort(_____ arr[], int _____, bool _____, int start = _____) {

    for (int i = _____; i < _____; i++) {

    }
}

int main() {
    int arr[6] = {8, 3, 6, 1, 9, 4};
    int size = 6;
    bool ascending;

    cout << "Original Array: " << endl;

    ____________________________;
```

```
    cout << "Sort in ascending order? "
         << "(1 for Yes, 0 for No): ";
    cin >> ascending;

    insertionSort(arr, size, ascending);

    cout << "Sorted Array: " << endl;

    ----------------------------;

    return 0;
}

// define displayArray function here
```

Sample Output 1

```
Original Array:
8 3 6 1 9 4
Sort in ascending order? (1 for Yes, 0 for No): 1
Sorted Array:
1 3 4 6 8 9
```

Sample Output 2

```
Original Array:
8 3 6 1 9 4
Sort in ascending order? (1 for Yes, 0 for No): 0
Sorted Array:
9 8 6 4 3 1
```